

design and train a convolutional neural network for Image classification for domain of your choice

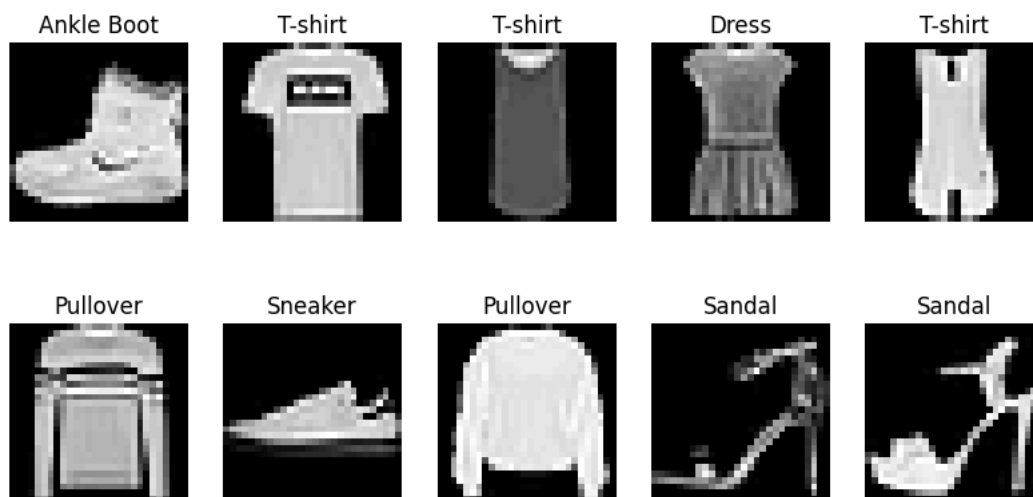
```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

```
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

```
class_names = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle Boot"]
```

```
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_train[i].reshape(28,28), cmap='gray')
    plt.title(class_names[np.argmax(y_train[i])])
    plt.axis('off')
plt.show()
```



```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    keras.layers.MaxPooling2D(2,2),

    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),

    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.4), # Regularization
    keras.layers.Dense(10, activation='softmax') # 10 output classes
])
```

```
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_3 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_4 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 128)	204,928
dropout_1 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)

```
history = model.fit(
    X_train, y_train,
    validation_data=(X_test, y_test),
    epochs=5,
    batch_size=64,
    verbose=1
)
```

```
Epoch 1/5
938/938 ————— 49s 52ms/step - accuracy: 0.8609 - loss: 0.3844 - val_accuracy: 0.8789 - val_loss: 0.3300
Epoch 2/5
938/938 ————— 83s 53ms/step - accuracy: 0.8845 - loss: 0.3211 - val_accuracy: 0.8905 - val_loss: 0.3018
Epoch 3/5
938/938 ————— 47s 50ms/step - accuracy: 0.8970 - loss: 0.2891 - val_accuracy: 0.9005 - val_loss: 0.2730
Epoch 4/5
938/938 ————— 83s 51ms/step - accuracy: 0.9045 - loss: 0.2654 - val_accuracy: 0.8968 - val_loss: 0.2810
Epoch 5/5
938/938 ————— 80s 49ms/step - accuracy: 0.9098 - loss: 0.2454 - val_accuracy: 0.9065 - val_loss: 0.2690
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
```

```
model.save("fashion_mnist_cnn.h5")
```

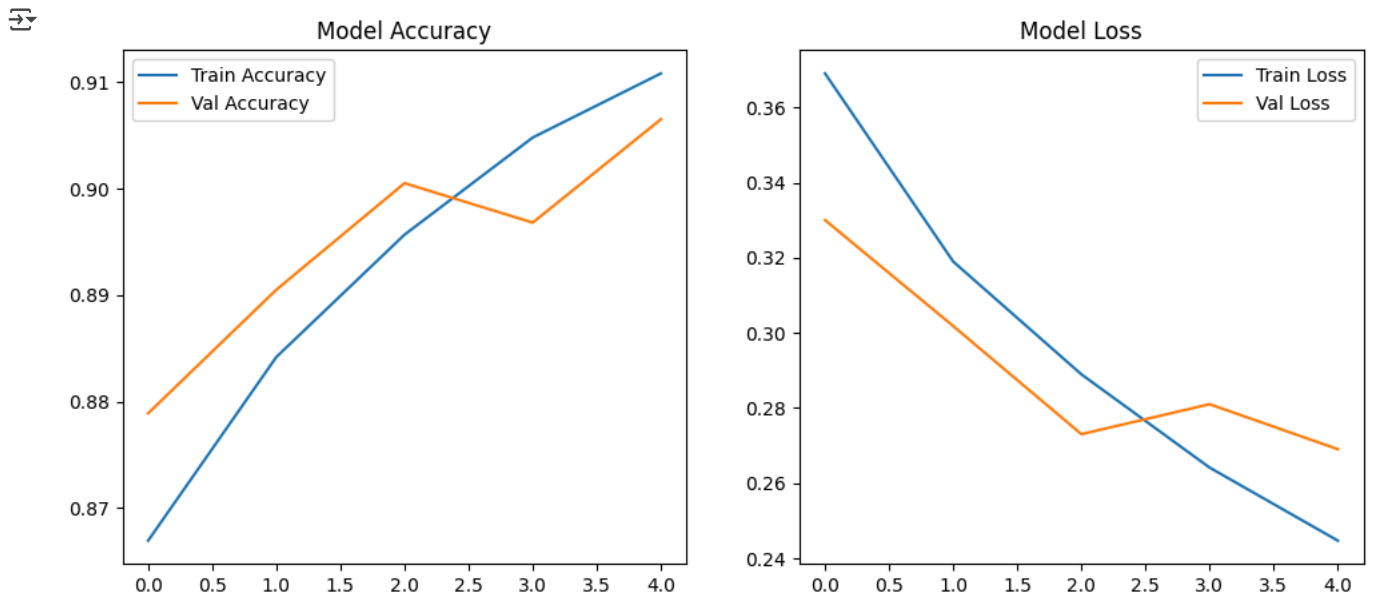
```
313/313 ————— 3s 8ms/step - accuracy: 0.9075 - loss: 0.2690
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is deprecated.
Test Accuracy: 0.9065
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.legend()
plt.title("Model Accuracy")

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title("Model Loss")

plt.show()
```



```

predictions = model.predict(X_test)
pred_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_test[i].reshape(28,28), cmap='gray')
    plt.title(f"Pred: {class_names[pred_labels[i]]}")
    plt.axis('off')
plt.show()

```

313/313 ————— 3s 8ms/step

